Rodion Zuban
SDEV200 - Software Development with Java
Timothy Newell

App: WAV Library
Link to GitHub Repository: https://github.com/rodionzuban/sdev200-final-project

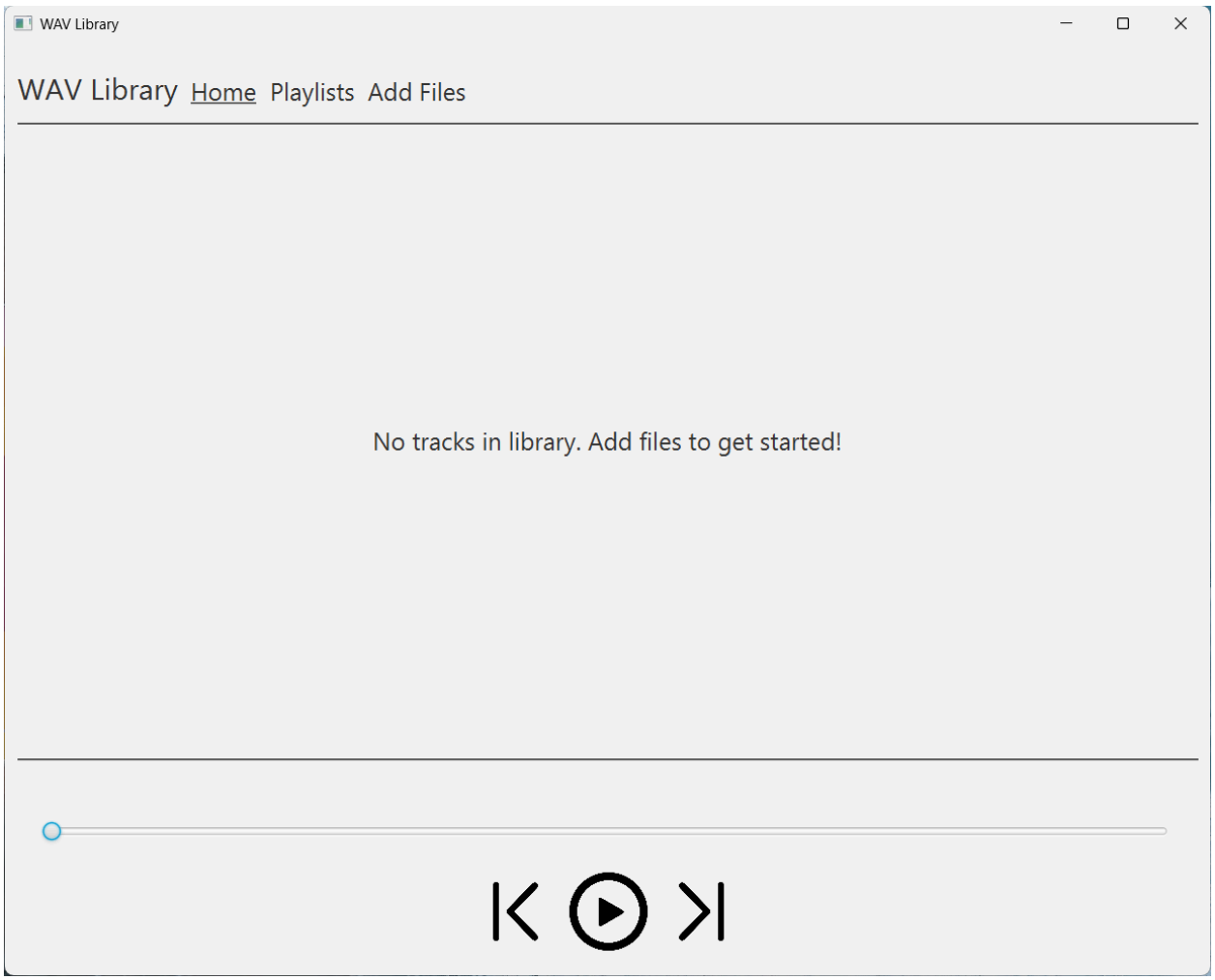
Description:
A JavaFX application to load, organize, and play .wav files stored on the user's machine.

Screens:
- Home Screen - displays all tracks loaded across the application
- Playlists Screen - displays all of the user's playlists
- Playlist Screen - shows a specific playlist, and allows the user to edit it
- Import Selection Screen - shows an option to import either a single .wav file or a folder of .wav files into the application
- Import Confirmation Screen - allows users to view their import batch (from the Import Selection Screen) and edit and remove tracks

The remainder of this document describes each screen individually and includes screenshots and input/output for events on the screen. A UML diagram for all non-UI classes is provided at the end.

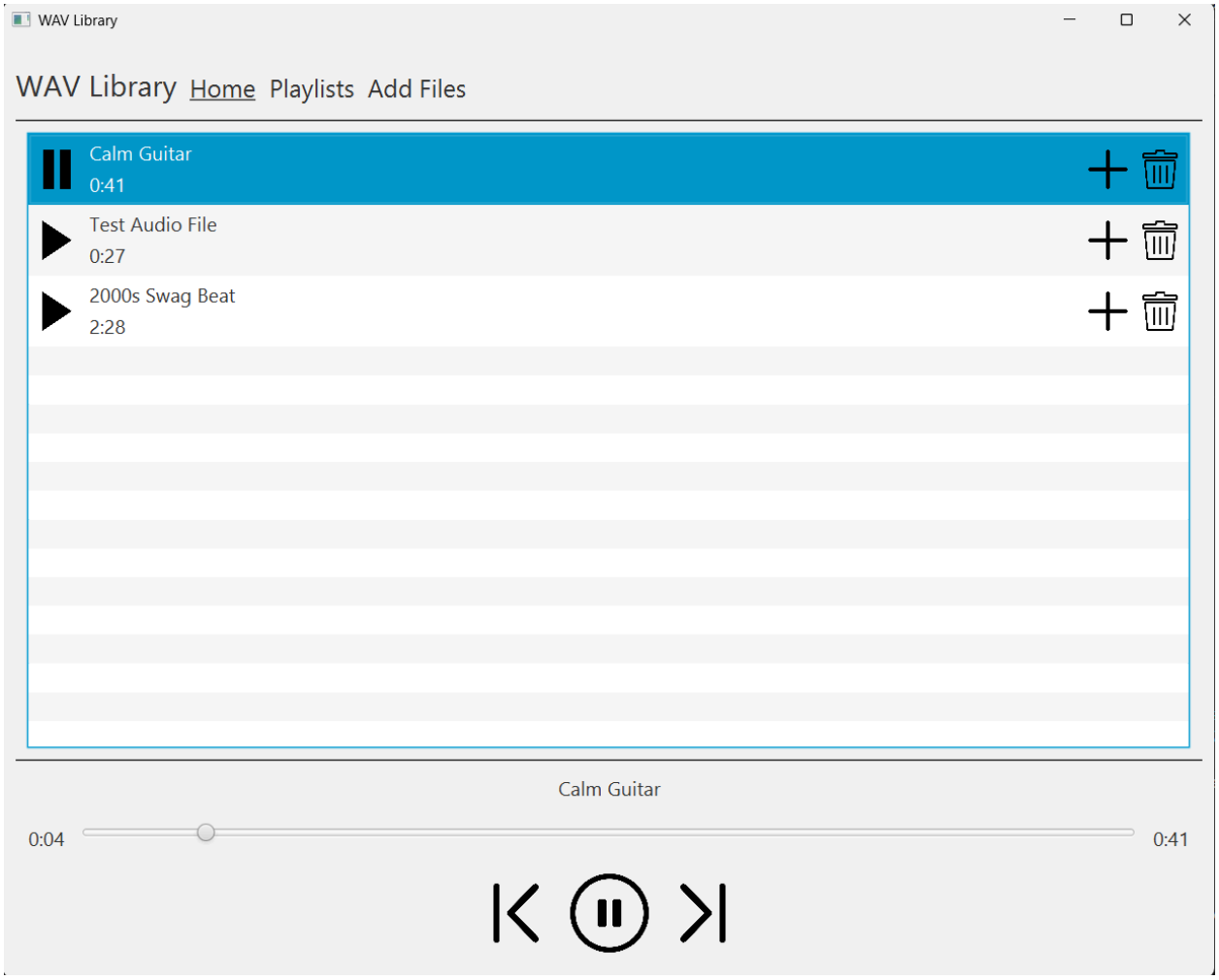
Home Screen

Home Screen on app load



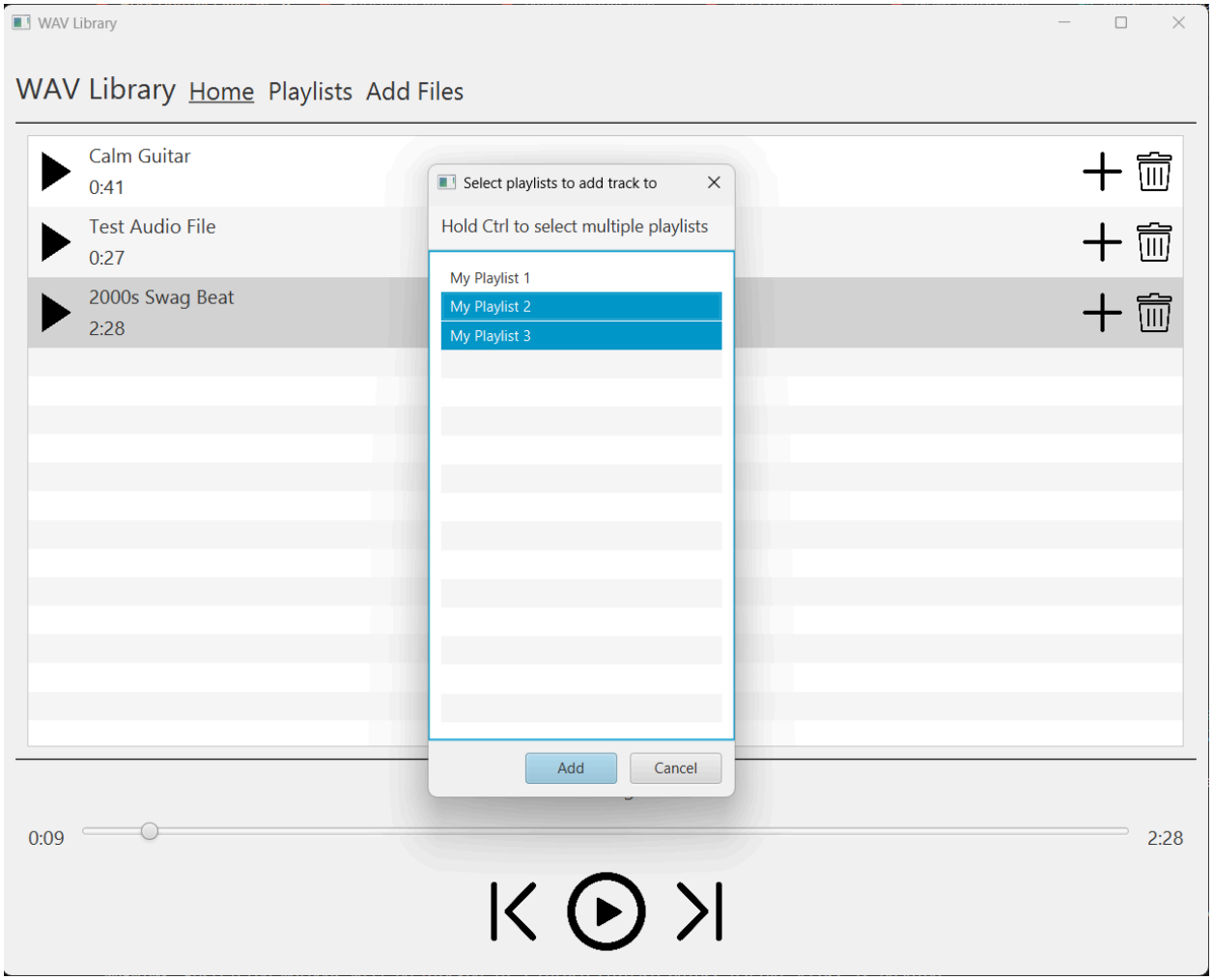
- No tracks have been loaded yet, so the application instructs the user to add files
- All playback components (play button, next, previous) are static and don't react when clicked
- Pressing the navigation options ("Home", "Playlists", "Add Files") navigates the user to the corresponding screen.

Home Screen with loaded tracks



- Dynamically lists all tracks that the user has loaded
    - When the user adds or deletes tracks, the list of tracks updates automatically
- Pressing the play button on a track shows its playback status (elapsed and total time, track title) at the bottom of the screen, and begins playback of the track
- Pressing the pause button in the track list or at the bottom of the screen pauses playback.
- Pressing the next button moves to the next track (wrapping around if at the last track)
- Pressing the previous button can do two things:
    - If the current track has been playing for more than 3 seconds, restart the track
    - Otherwise, moves to the previously played track.
- Pressing the trash can icon removes the track globally (including from all playlists)
- Pressing the "+" icon shows the next described screen

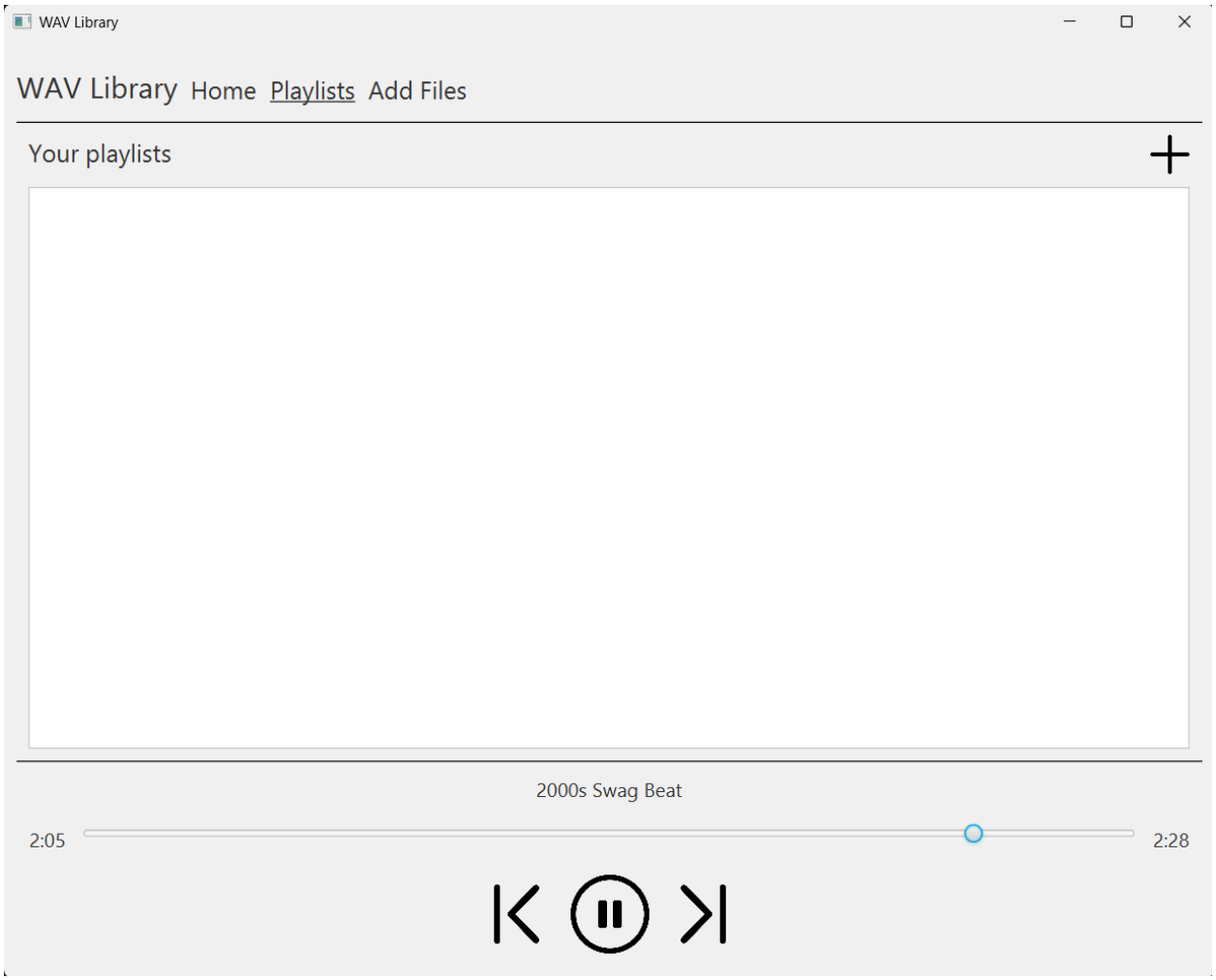
Home Screen with playlist selection dialog



- A selection dialog is shown, with all of the user's playlists listed
    - Playlists that already have the selected track are automatically selected in the dialog, and unselecting them removes the track from that playlist
- The user can select playlists to add the track to, and hold down the Ctrl key to select multiple playlists
- When the user presses the "Add" button, the track is added to all playlists selected by the user, and removed from all playlists not selected by the user. The dialog is closed
    - Essentially, all playlists selected now contain the track, and all others don't
- Pressing "Cancel" closes the dialog

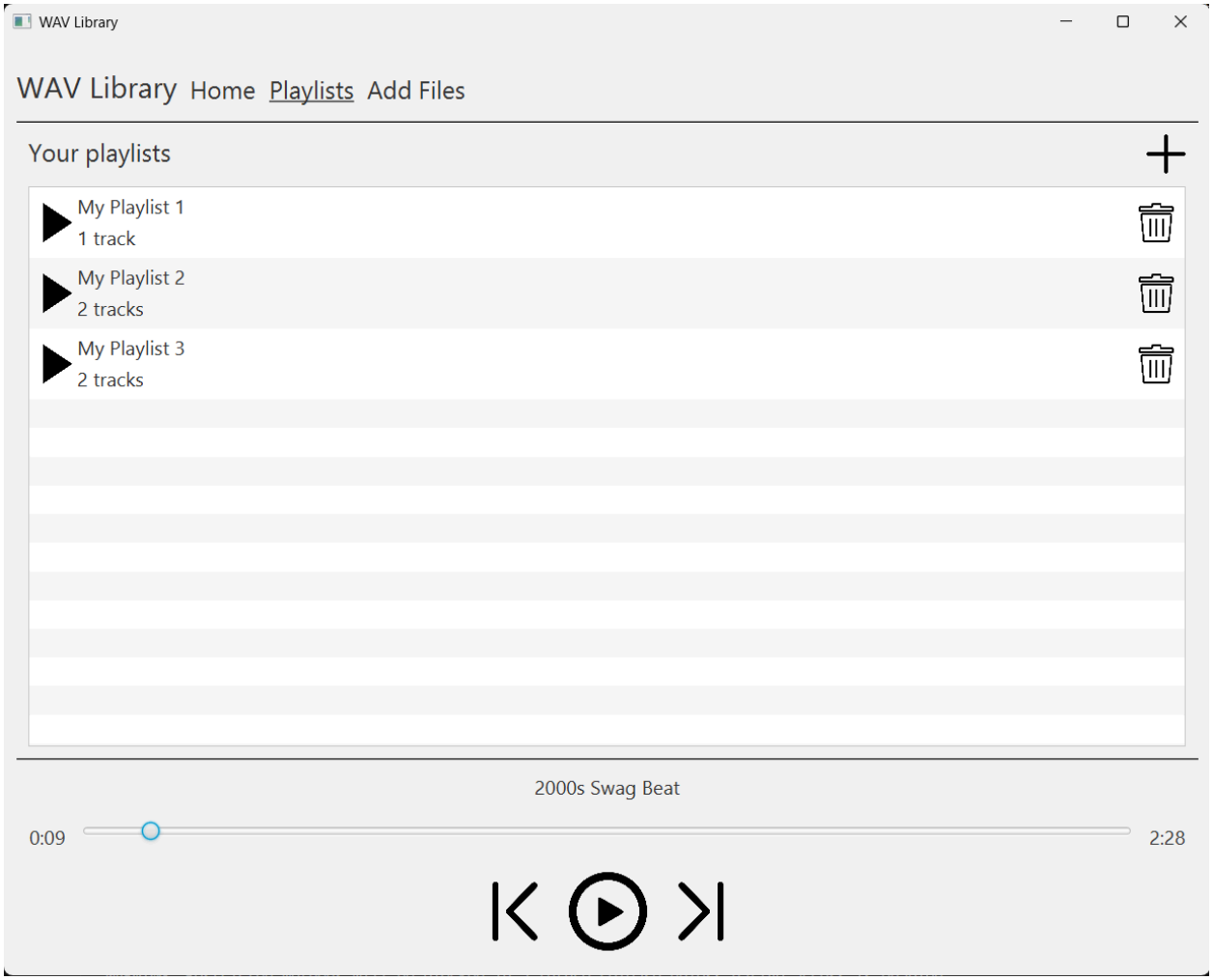
Playlists Screen

Blank Playlists Screen



- Playback components at the bottom of the screen behave as described in the Home Screen section
- Pressing the "+" icon adds a new playlist to the app, as shown in the next screen

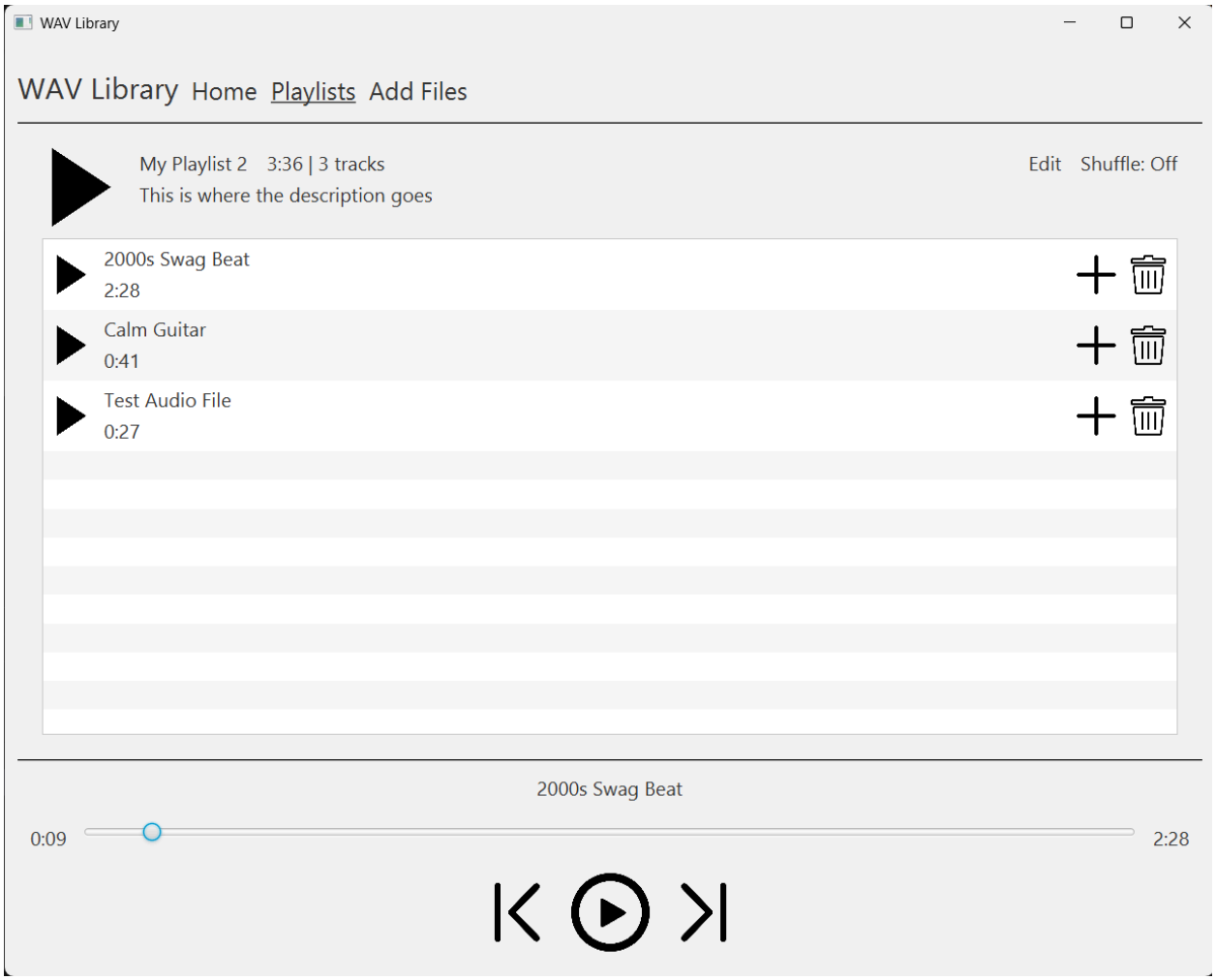
Playlists Screen populated with playlists



- Playback components at the bottom of the screen behave as described in the Home Screen section
- The "+" icon behaves as described in the previous section
- All of the user's playlists are listed dynamically, updating when a playlist is created or removed. Playlist information, like names and track counts are updated dynamically as well.
- Pressing the play icon on a playlist starts playback of that playlist, starting with the first track in the playlist if it hasn't been played yet, or with the "current" track in the playlist
    - Pressing pause pauses playback of the playlist
- Pressing the name or numbers of the playlist redirects the user to that playlist's Playlist Screen.
- Pressing the trash icon removes the playlist, but playback continues using the existing track queue even after the playlist is deleted.

Playlist Screen

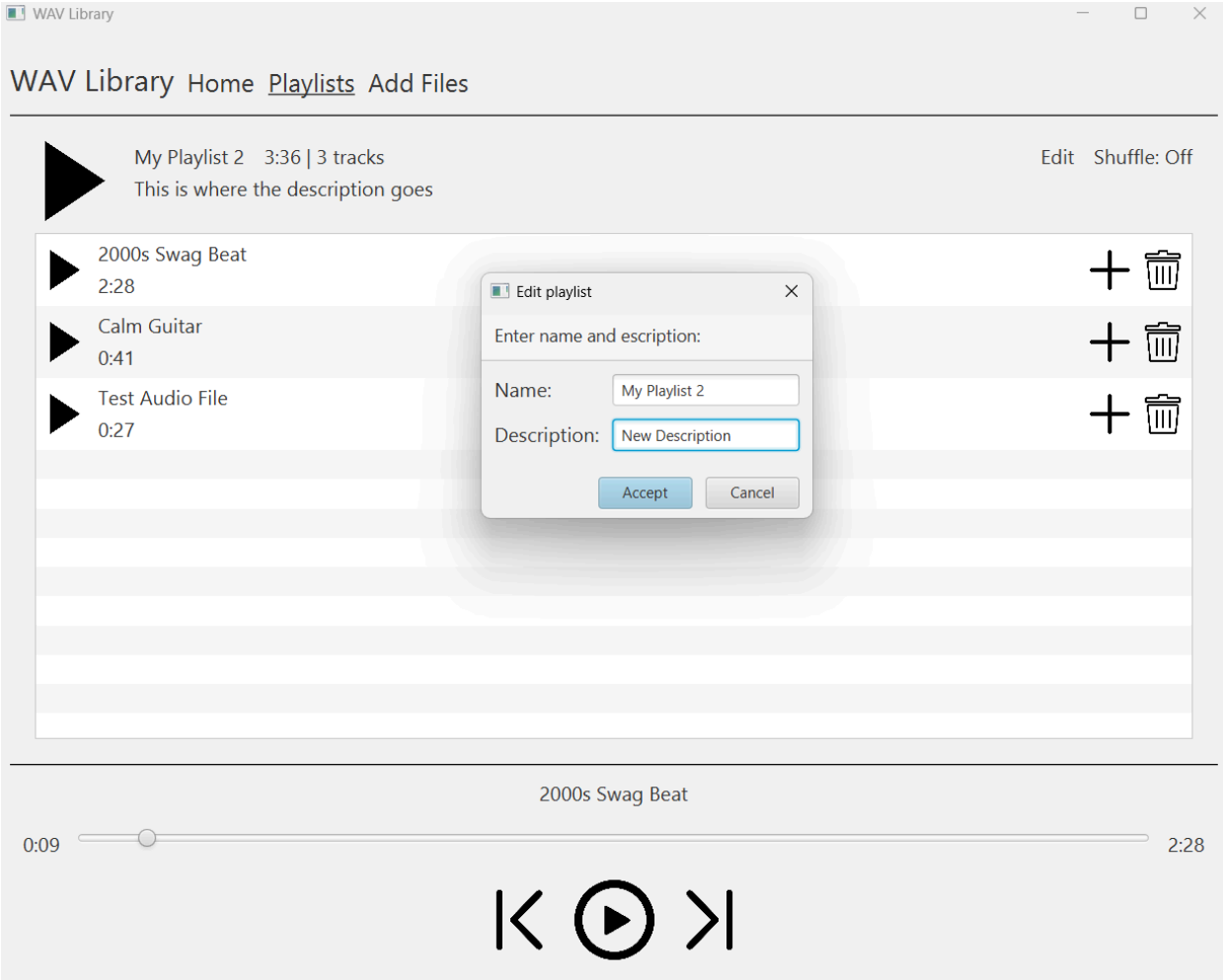
Playlist Screen with tracks



- Playback components at the bottom of the screen behave as described in the Home Screen section
- The playlist name, description, length in time, and number of tracks are displayed dynamically at the top of the screen
- Pressing the large play icon plays the playlist, exactly as described in the previous section, adhering to the playlist order (or shuffled order), or playing the first track, if the playlist hasn't been shuffled yet
- Pressing the smaller play button next to each of the tracks plays that track as described in the Home Screen section, except it sets the playback queue to the playlist instead of the global collection of tracks
- The "+" icon behaves exactly as described in the home screen
- The trash icon removes the track from the playlist, but keeps it in playback if currently playing

- The “Shuffle: Off/On” text toggles shuffle mode. Pressing it either shuffles the track queue or resets it to playing in the standard order of tracks
- Pressing the “edit” text allows the user to edit the playlist name and description, as described in the next screen.

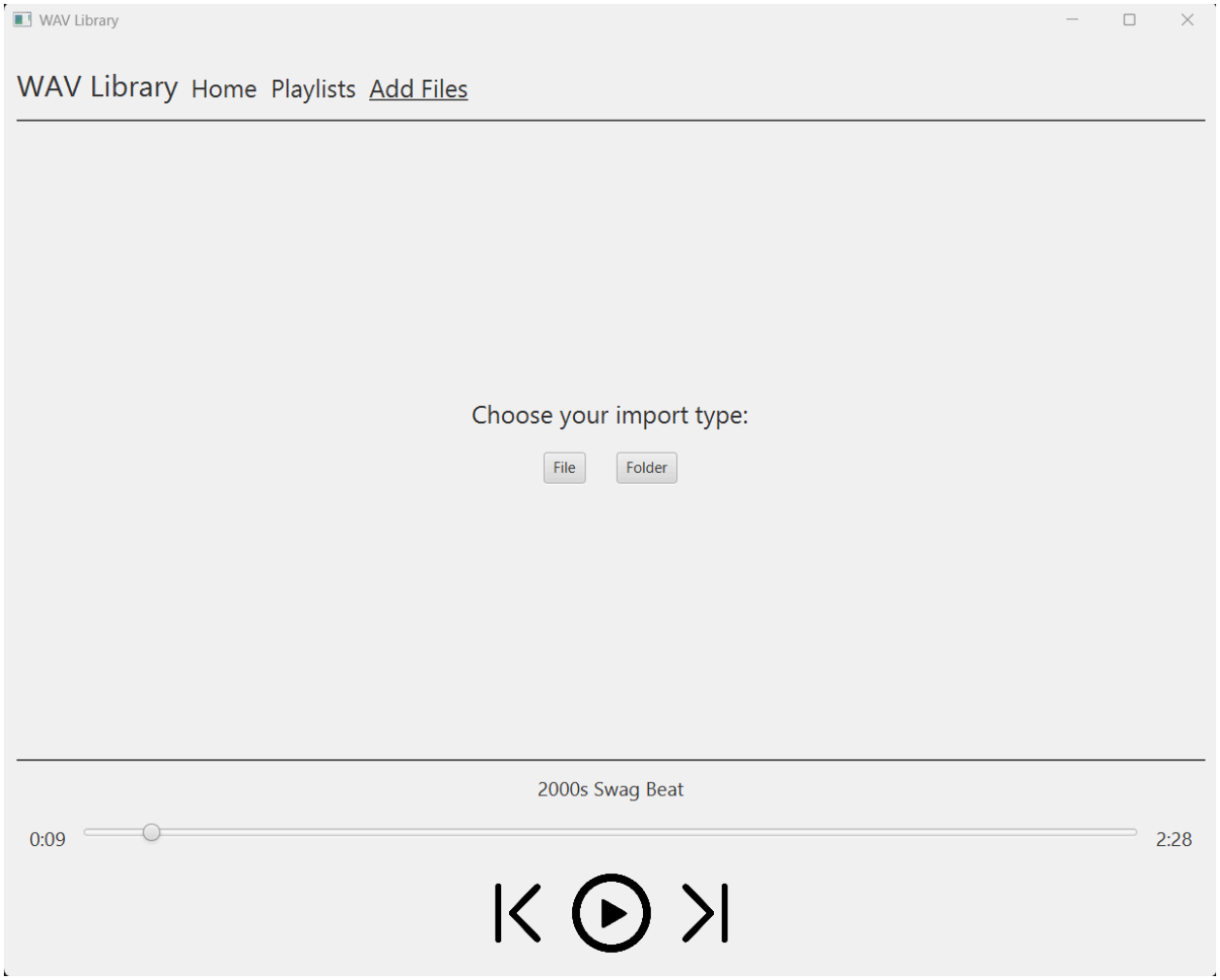
Playlist Screen with editing dialog



- The user is prompted with two text boxes to enter a new name and/or description for the playlist
- The values in the text boxes default to the current values for their corresponding properties
- Pressing “Accept” sets the name and description to what the user inputted, and updates that information across the application.
- Pressing “Cancel” closes the dialog.

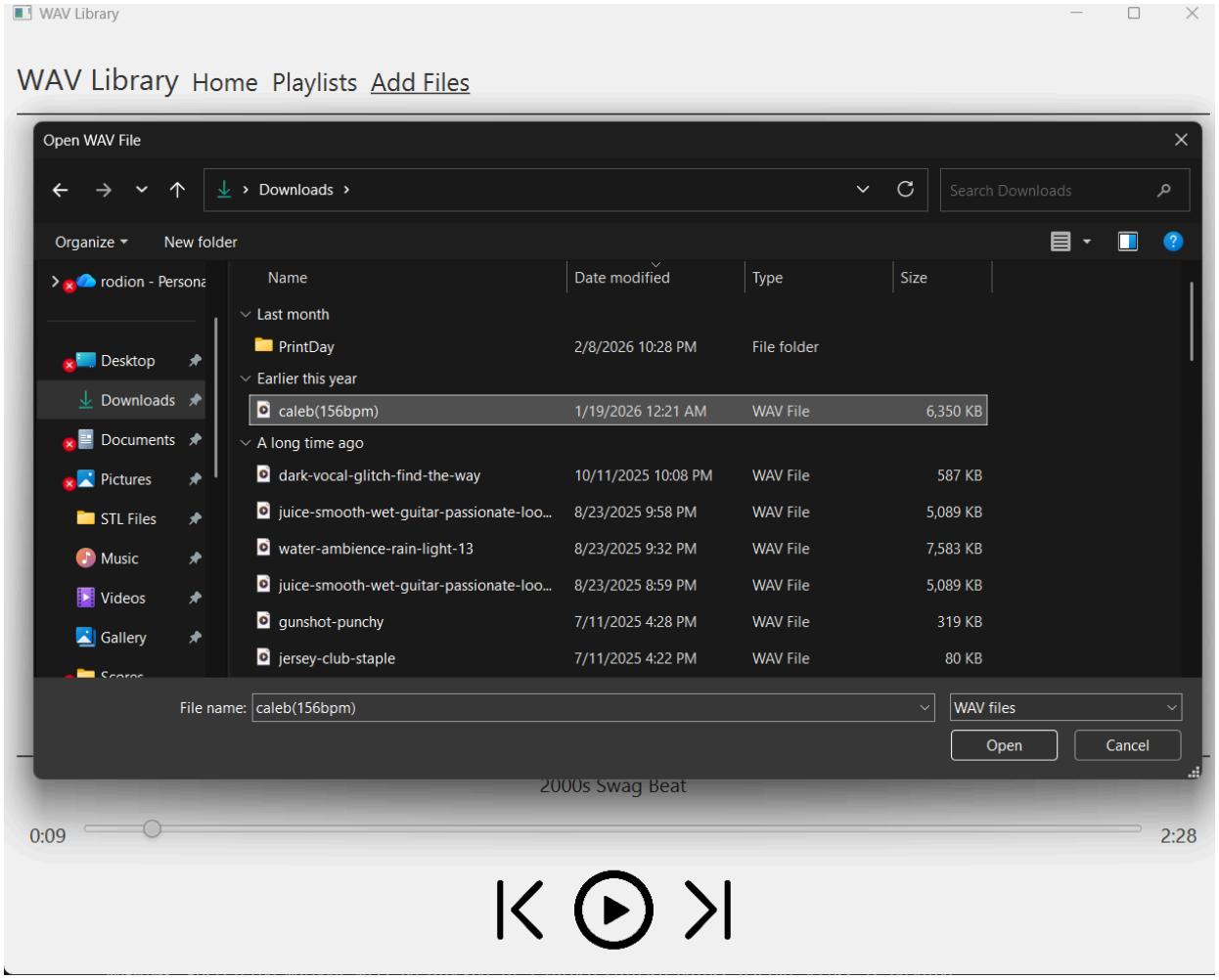
Import Selection Screen

Import selection prompt on Import Selection Screen



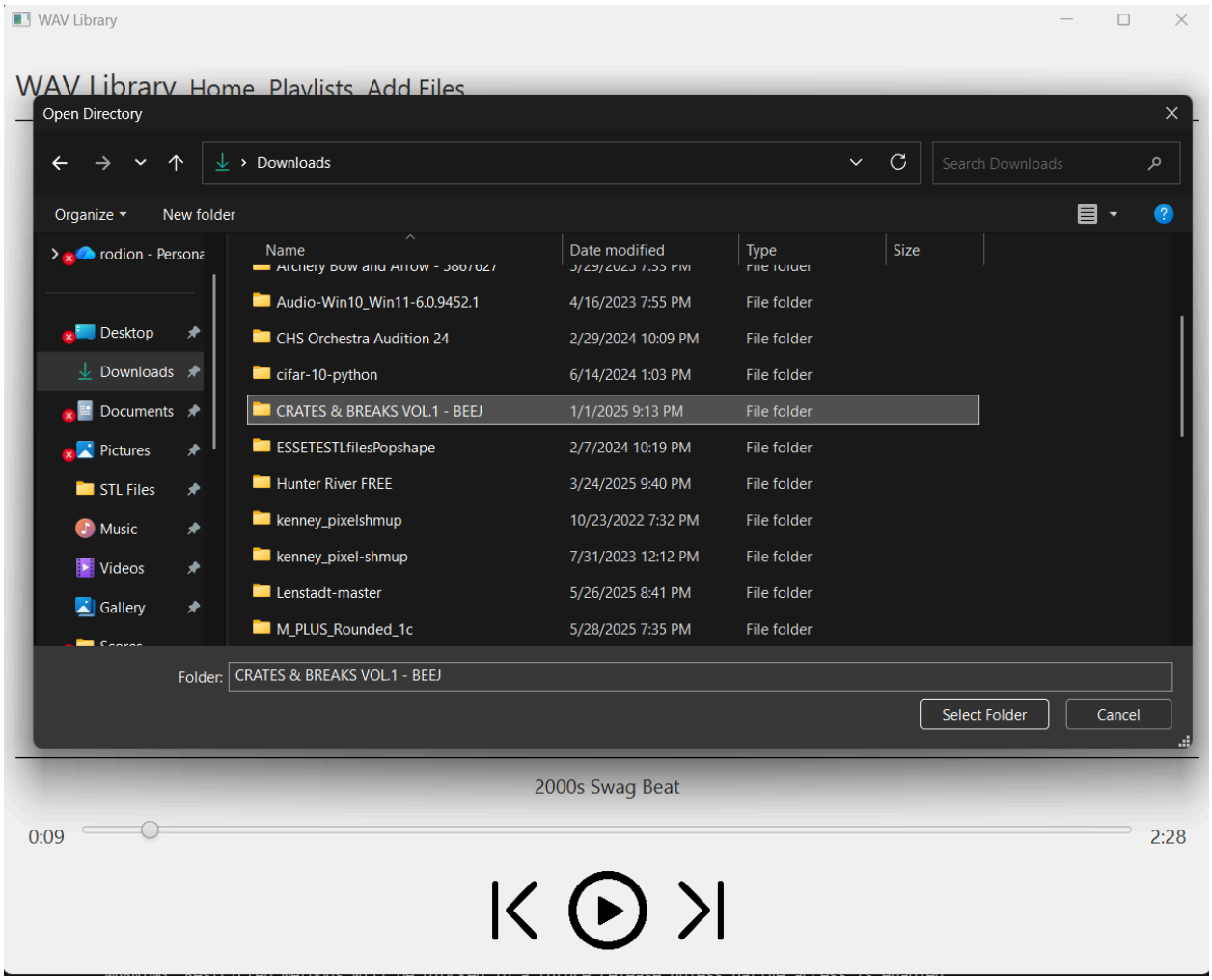
- Playback components at the bottom of the screen behave as described in the Home Screen section
- Clicking “File” prompts the user with a file selection window, and clicking “folder” prompts the user with a folder selection window, as described in the following sections.

File selection window from import selection screen



- The user can browse their files, and can select only .wav files from their files.
- Selecting a file, then pressing “Open” loads the corresponding .wav file into the Import Confirmation screen
- Pressing “cancel” closes the selection window

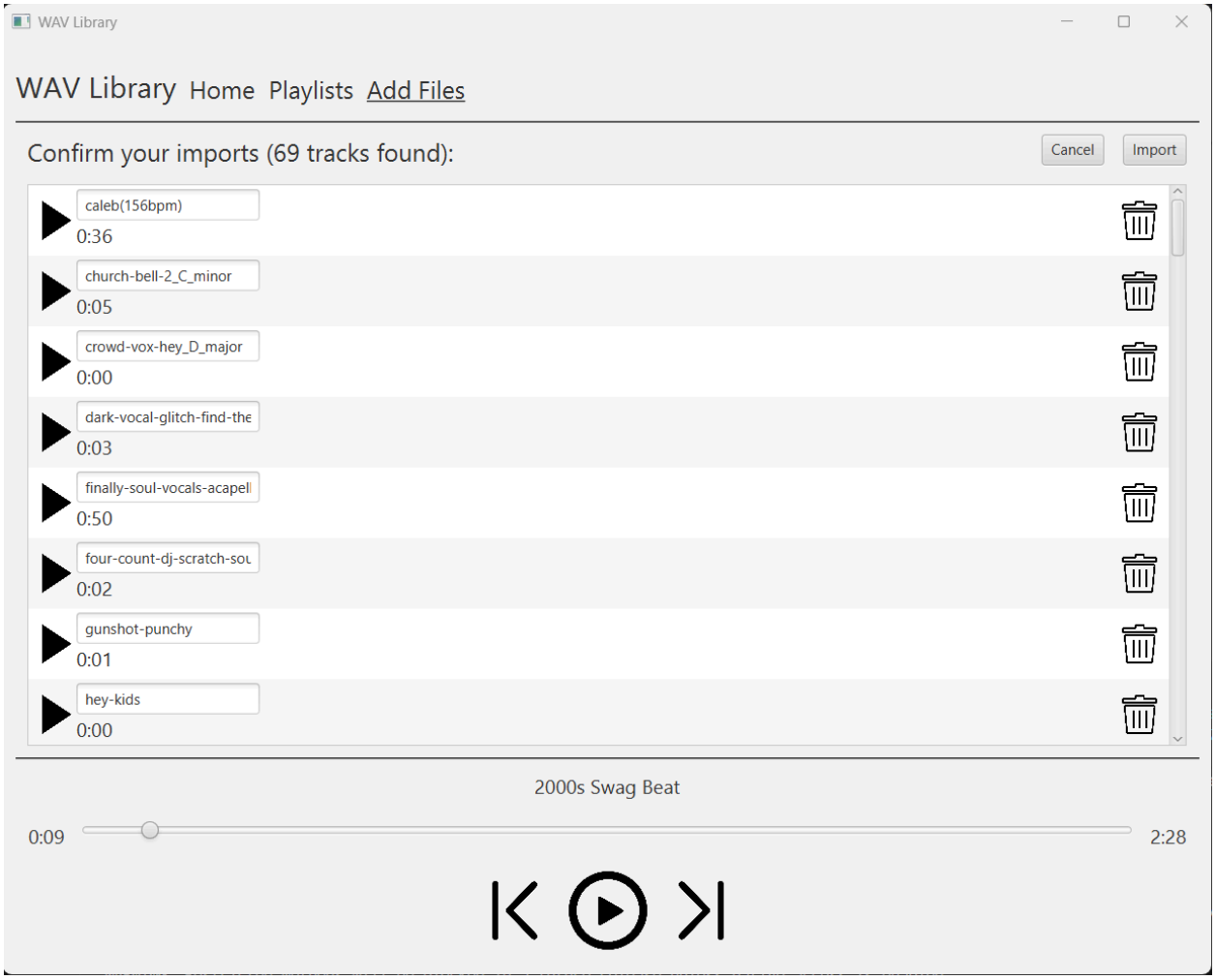
Folder selection window from import selection screen



- The user can browse their files, and can select only directories to be imported into the application
- Selecting a folder, then pressing “Select Folder” loads the folder’s .wav files into the Import Confirmation screen
    - The program recursively searches through all subdirectories in the folder, and picks only .wav files to load in
- Pressing “Cancel” closes the selection window

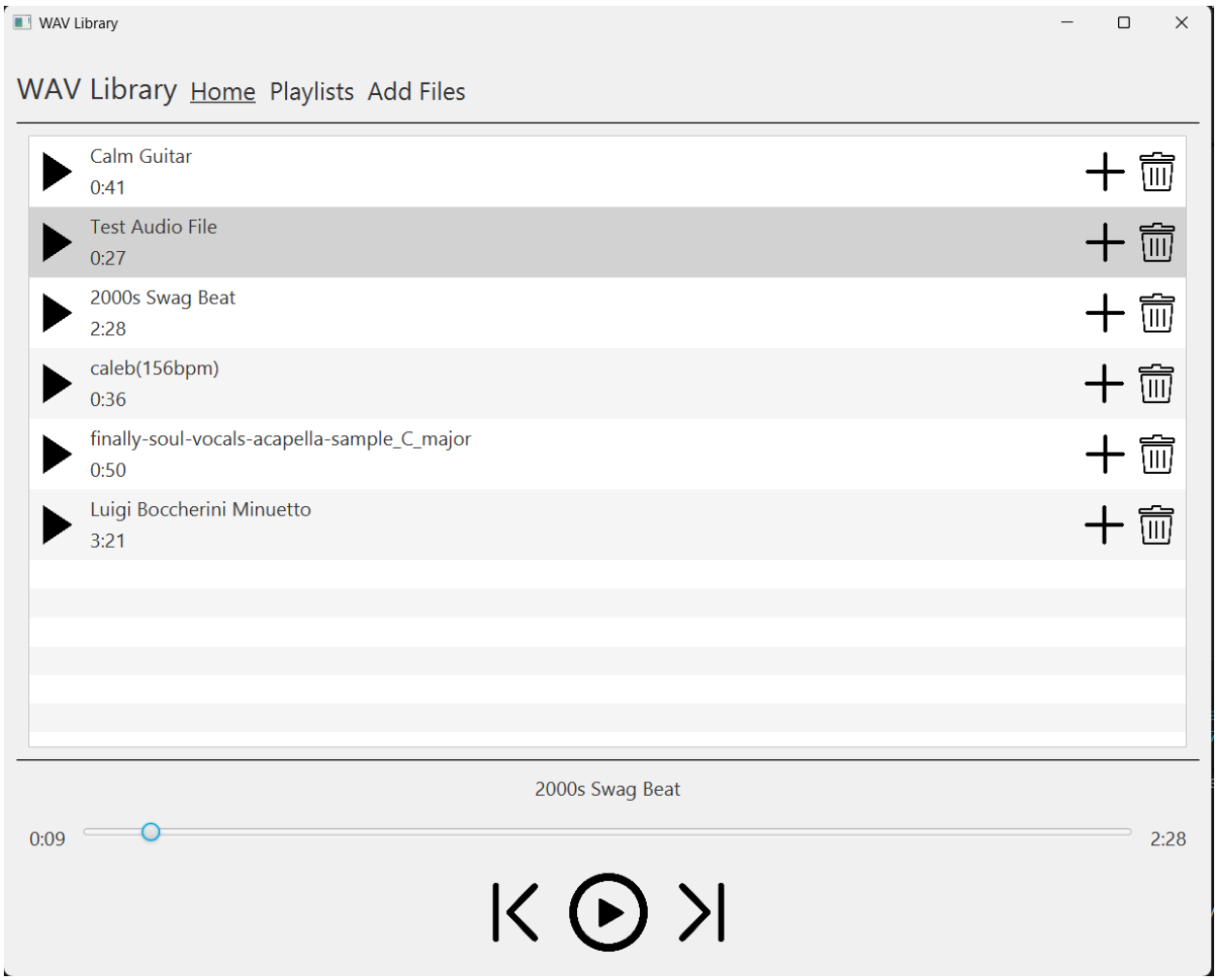
Import Selection Screen

Import confirmation screen with loaded tracks

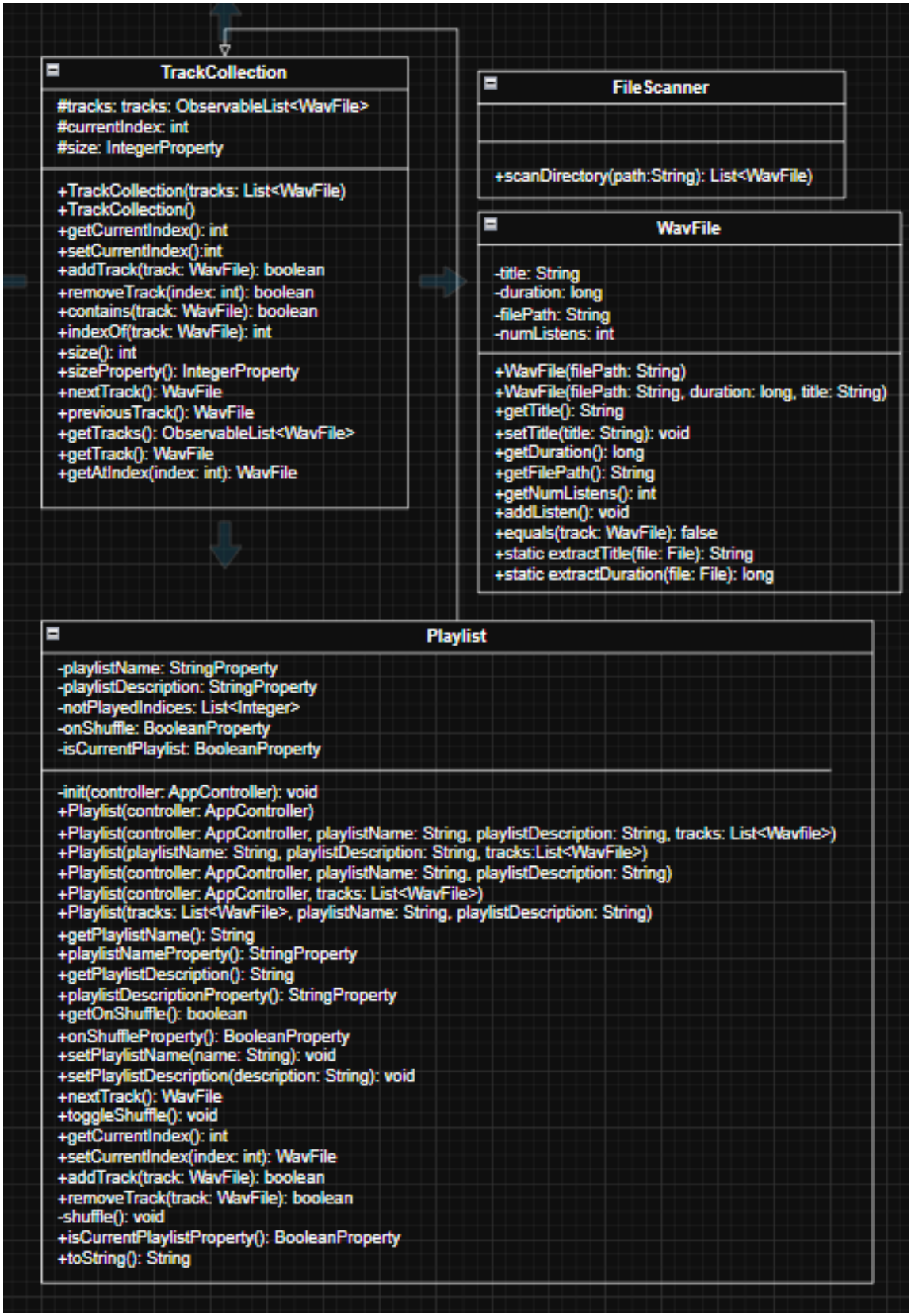


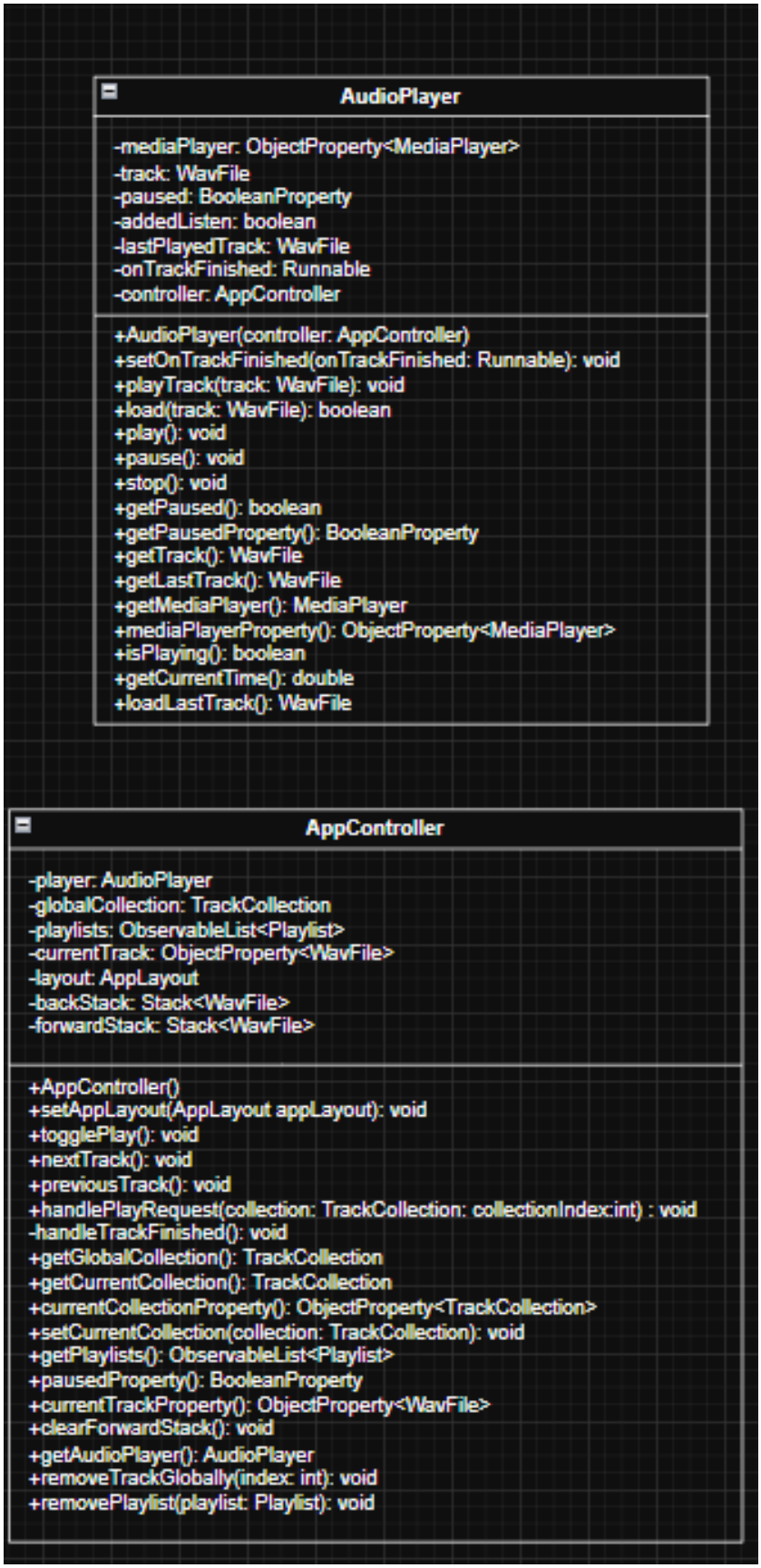
- All of the .wav files from the selected directory (or the single chosen .wav file) are displayed here
- Pressing the play button on a specific track begins playback of that track and sets the playback collection to the collection of tracks in the import batch, which will play in standard order
- The user can rename the tracks in the text box provided for each track, and the name the user chooses will be the name used for the track throughout the application
- Pressing the trash icon removes the track from the import batch, and removes it from the list of tracks shown on the screen
- Pressing the “Import” button adds all of the tracks, in the order shown on the screen, to the global collection of tracks (as shown in the Home Screen)
    - Tracks that match in filepath to existing tracks (duplicate tracks) are not added

- Pressing the cancel button closes this screen (although playback of the import batch can continue) and redirects the user back to the Import Selection Screen
- Playback components at the bottom of the screen behave as described in the Home Screen section
- Example of tracks imported into the application as shown on the home screen:



UML


TrackCollection
#tracks: tracks: ObservableList<WavFile>
#currentIndex: int
#size: IntegerProperty
+TrackCollection(tracks: List<WavFile)
+TrackCollection()
+getCurrentIndex(): int
+setCurrentIndex():int
+addTrack(track: WavFile): boolean
+removeTrack(index: int): boolean
+contains(track: WavFile): boolean
+indexOf(track: WavFile): int
+size(): int
+sizeProperty(): IntegerProperty
+nextTrack(): WavFile
+previousTrack(): WavFile
+getTracks(): ObservableList<WavFile>
+getTrack(): WavFile
+getAtIndex(index: int): WavFile
FileScanner
+scanDirectory(path:String): List<WavFile)
WavFile
-title: String
-duration: long
-filePath: String
-numListens: int
+WavFile(filePath: String)
+WavFile(filePath: String, duration: long, title: String)
+getTitle(): String
+setTitle(title: String): void
+getDuration(): long
+getFilePath(): String
+getNumListens(): int
+addListen(): void
+equals(track: WavFile): false
+static extractTitle(file: File): String
+static extractDuration(file: File): long
Playlist
-playlistName: StringProperty
-playlistDescription: StringProperty
-notPlayedIndices: List<Integer>
-onShuffle: BooleanProperty
-isCurrentPlaylist: BooleanProperty
-init(controller: AppController): void
+Playlist(controller: AppController)
+Playlist(controller: AppController, playlistName: String, playlistDescription: String, tracks: List<Wavfile>)
+Playlist(playlistName: String, playlistDescription: String, tracks:List<WavFile>)
+Playlist(controller: AppController, playlistName: String, playlistDescription: String)
+Playlist(controller: AppController, tracks: List<WavFile>)
+Playlist(tracks: List<WavFile>, playlistName: String, playlistDescription: String)
+getPlaylistName(): String
+playlistNameProperty(): StringProperty
+getPlaylistDescription(): String
+playlistDescriptionProperty(): StringProperty
+getOnShuffle(): boolean
+onShuffleProperty(): BooleanProperty
+setPlaylistName(name: String): void
+setPlaylistDescription(description: String): void
+nextTrack(): WavFile
+toggleShuffle(): void
+getCurrentIndex(): int
+setCurrentIndex(index: int): WavFile
+addTrack(track: WavFile): boolean
+removeTrack(track: WavFile): boolean
-shuffle(): void
+isCurrentPlaylistProperty(): BooleanProperty
+toString(): String


AudioPlayer
-mediaPlayer: ObjectProperty<MediaPlayer>
-track: WavFile
-paused: BooleanProperty
-addedListen: boolean
-lastPlayedTrack: WavFile
-onTrackFinished: Runnable
-controller: AppController
+AudioPlayer(controller: AppController)
+setOnTrackFinished(onTrackFinished: Runnable): void
+playTrack(track: WavFile): void
+load(track: WavFile): boolean
+play(): void
+pause(): void
+stop(): void
+getPaused(): boolean
+getPausedProperty(): BooleanProperty
+getTrack(): WavFile
+getLastTrack(): WavFile
+getMediaPlayer(): MediaPlayer
+mediaPlayerProperty(): ObjectProperty<MediaPlayer>
+isPlaying(): boolean
+getCurrentTime(): double
+loadLastTrack(): WavFile
AppController
-player: AudioPlayer
-globalCollection: TrackCollection
-playlists: ObservableList<Playlist>
-currentTrack: ObjectProperty<WavFile>
-layout: AppLayout
-backStack: Stack<WavFile>
-forwardStack: Stack<WavFile>
+AppController()
+setAppLayout(AppLayout appLayout): void
+togglePlay(): void
+nextTrack(): void
+previousTrack(): void
+handlePlayRequest(collection: TrackCollection: collectionIndex:int) : void
-handleTrackFinished(): void
+getGlobalCollection(): TrackCollection
+getCurrentCollection(): TrackCollection
+currentCollectionProperty(): ObjectProperty<TrackCollection>
+setCurrentCollection(collection: TrackCollection): void
+getPlaylists(): ObservableList<Playlist>
+pausedProperty(): BooleanProperty
+currentTrackProperty(): ObjectProperty<WavFile>
+clearForwardStack(): void
+getAudioPlayer(): AudioPlayer
+removeTrackGlobally(index: int): void
+removePlaylist(playlist: Playlist): void